Inyección de datos

Agustín C. Caminero Herráez
Depto. de Sistemas de Comunicación y Control.
UNED
accaminero@scc.uned.es / @accaminero

ÍNDICE

- Herramientas para inyectar datos.
- Flume.
- Sqoop.

OBJETIVOS

- Conocer distintos casos de uso de inyección de datos.
- Conocer distintas herramientas para realizar inyección de datos, sus características y los casos de uso en que están indicadas.

HERRAMIENTAS PARA LA INYECCIÓN DE DATOS

- Hadoop NO es un sistema en el que las transferencias de datos se realicen en forma de trozos de información pequeños y discretos, como sería por ejemplo en un sistema de reservas de una compañía aérea.
- Normalmente, los datos se cargan en Hadoop realizando transferencias de grandes cantidades de datos inyectados de fuentes externas, tales como ficheros de log, transferencias de bases de datos relacionales, o repositorios de datos como el Banco Mundial.
- Los datos no se quedan en Hadoop, sino que siguen su curso hacia otras herramientas (visualización, BI…).

HERRAMIENTAS PARA LA INYECCIÓN DE DATOS

- Diferentes casos de uso:
  - Datos basados en eventos.
    - Ej: ficheros de log, datos de redes sociales, …
  - Datos de fuentes estructuradas.
    - Ej: base de datos relacionales
  - …

HERRAMIENTAS PARA LA INYECCIÓN DE DATOS

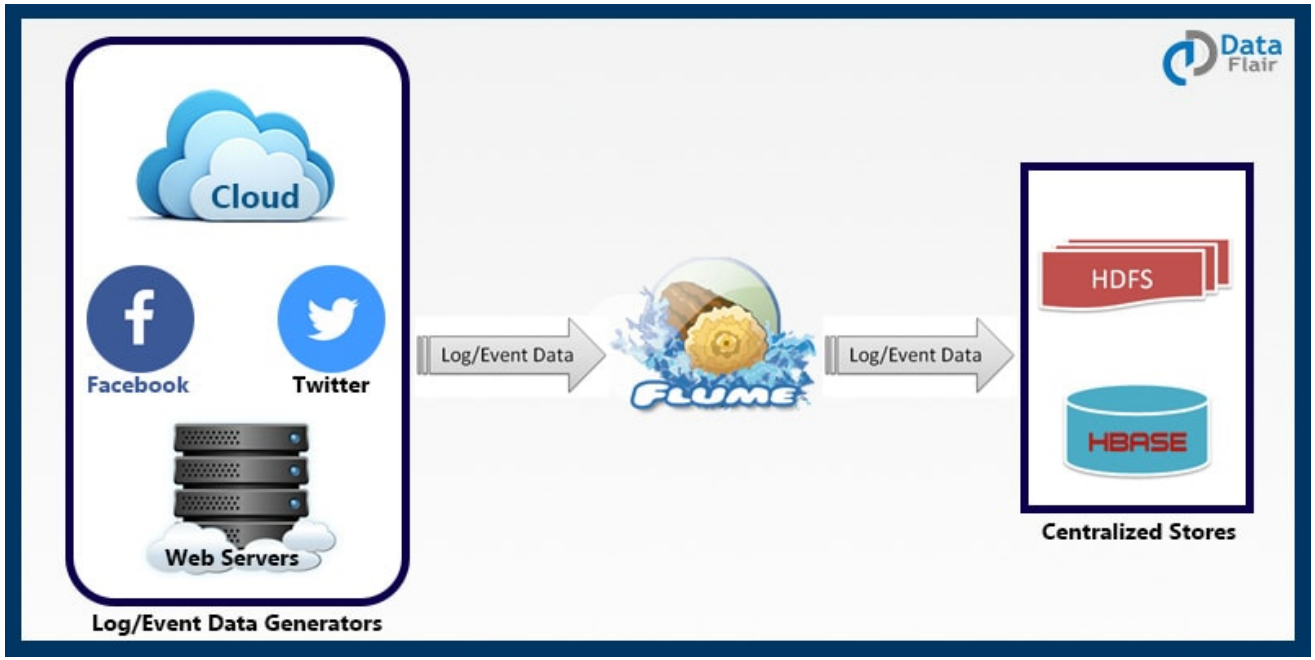
- Diferentes casos de uso:
  - Datos basados en eventos.
    - Ej: ficheros de log, datos de redes sociales, ...
  - Datos de fuentes estructuradas.
    - Ej: base de datos relacionales
  - …





IMPORTANDO DATOS DE EVENTOS: FLUME

- Flume es una herramienta para tranferir datos desde varias fuentes hasta los diferentes sistemas de almacenamiento que se encuentran en el ecosistema de Hadoop, como por ejemplo HDFS y HBase.

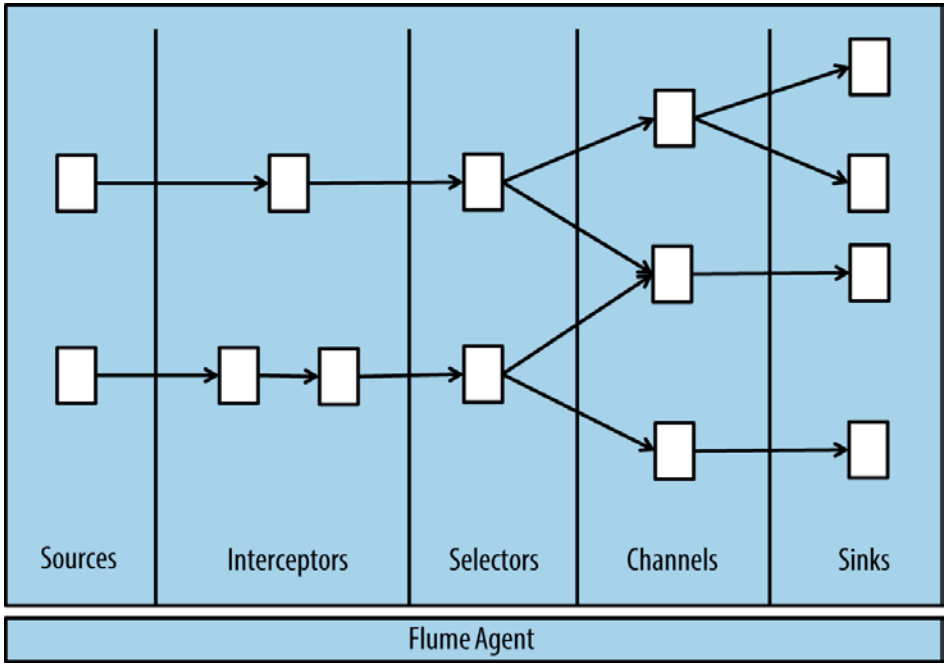


ARQUITECTURA DE FLUME

- La unidad de desarrollo de Flume más sencilla es el *agente Flume*.
  - Un agente es una aplicación Java que recibe o genera datos y los almacena hasta que se los pasa al siguiente agente o se escriben en el sistema de almacenamiento final (HDFS, Hbase).
- Podemos definir *flujos* de datos en los cuales un agente recibe datos generados por otro agente y se los envía a otro agente más.
- Al tener una serie de agentes de flume recibiendo datos desde las fuentes de datos, y escribiéndolos a HDFS o Hbase (bien directamente o bien a través de otros agentes), podemos escalar el número de fuentes de datos y la cantidad de datos que se escriben simplemente añadiendo más agentes.

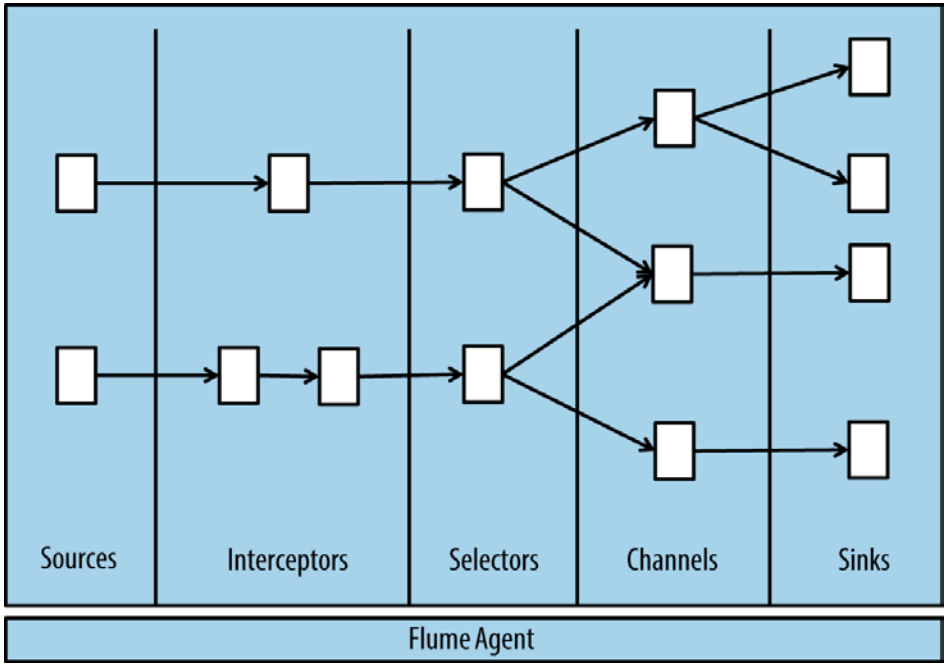
AGENTE FLUME

- Sources:
  - Componentes activos que reciben datos de otra aplicación que los está produciendo.
- Interceptors:
  - Permiten que los eventos sean interceptados y modificados al vuelo (filtrados, enriquecidos, …)



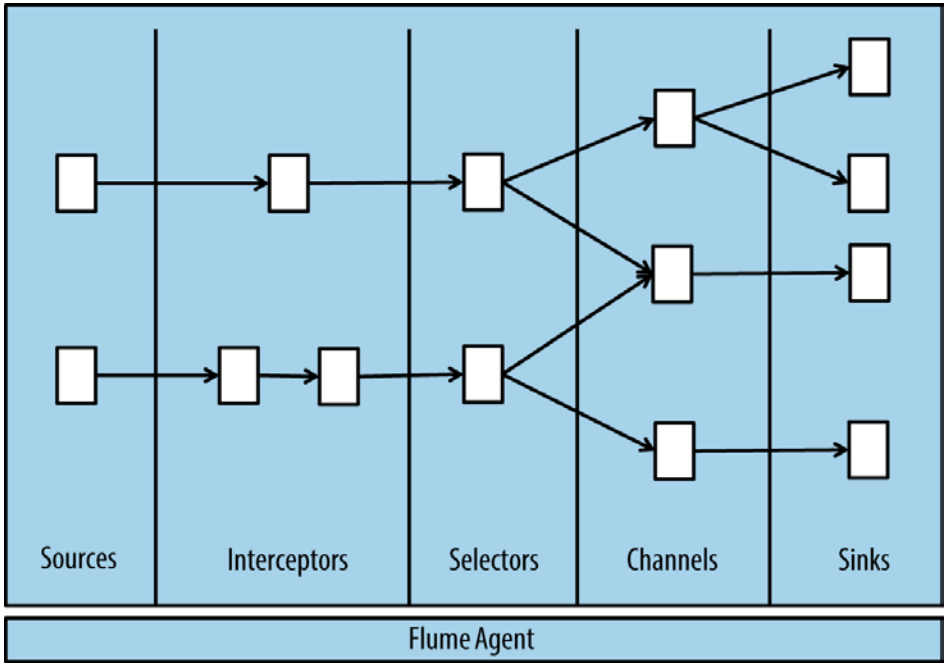
AGENTE FLUME

- Selectors:
  - Enrutan los eventos. Deciden a cuál/es de los canales se envía un evento.
- Channels:
  - Son generalmente componentes pasivos en los que se almacenan los datos recibidos por el agente pero que aún no han sido escritos en el sistema de almacenamiento o bien en otro agente.
  - Múltiples sources pueden escribir en el mismo channel y múltiples sinks pueden leer el mismo channel.

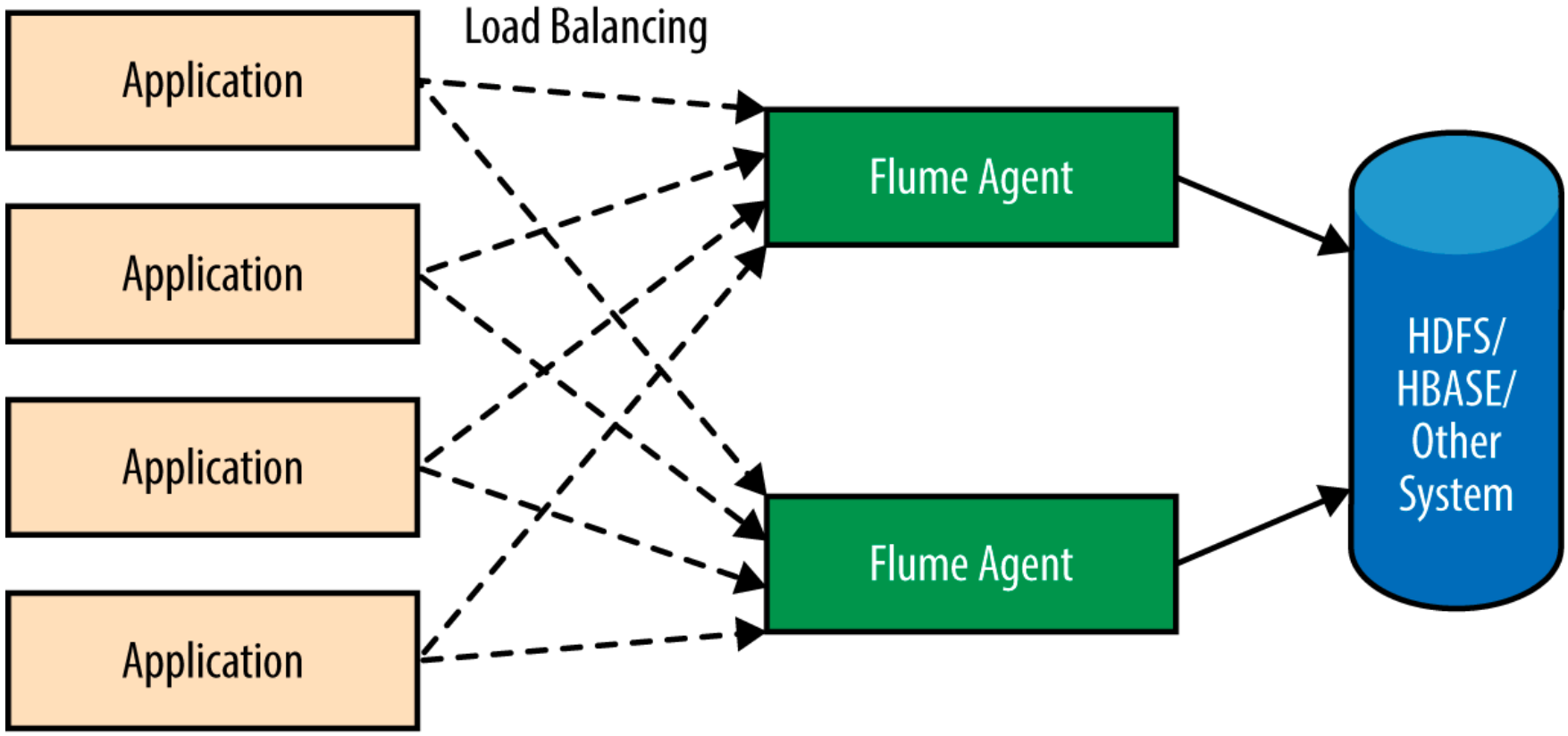


AGENTE FLUME

- Sinks:
  - Sondean el canal al que se encuentran conectados (solamente uno) para leer y escribir eventos.
  - Una vez que un dato se encuentra en el siguiente agente del flujo o bien en el sistema de almacenamiento final, los sinks informan al canal para que elimine esos datos.

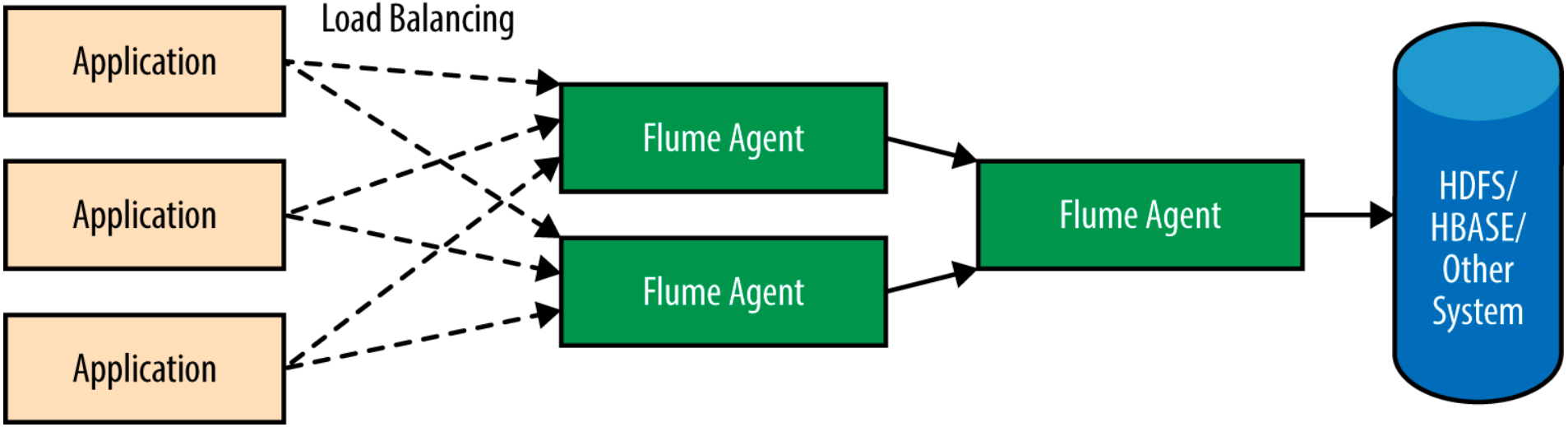


FLUJOS DE DATOS DE FLUME

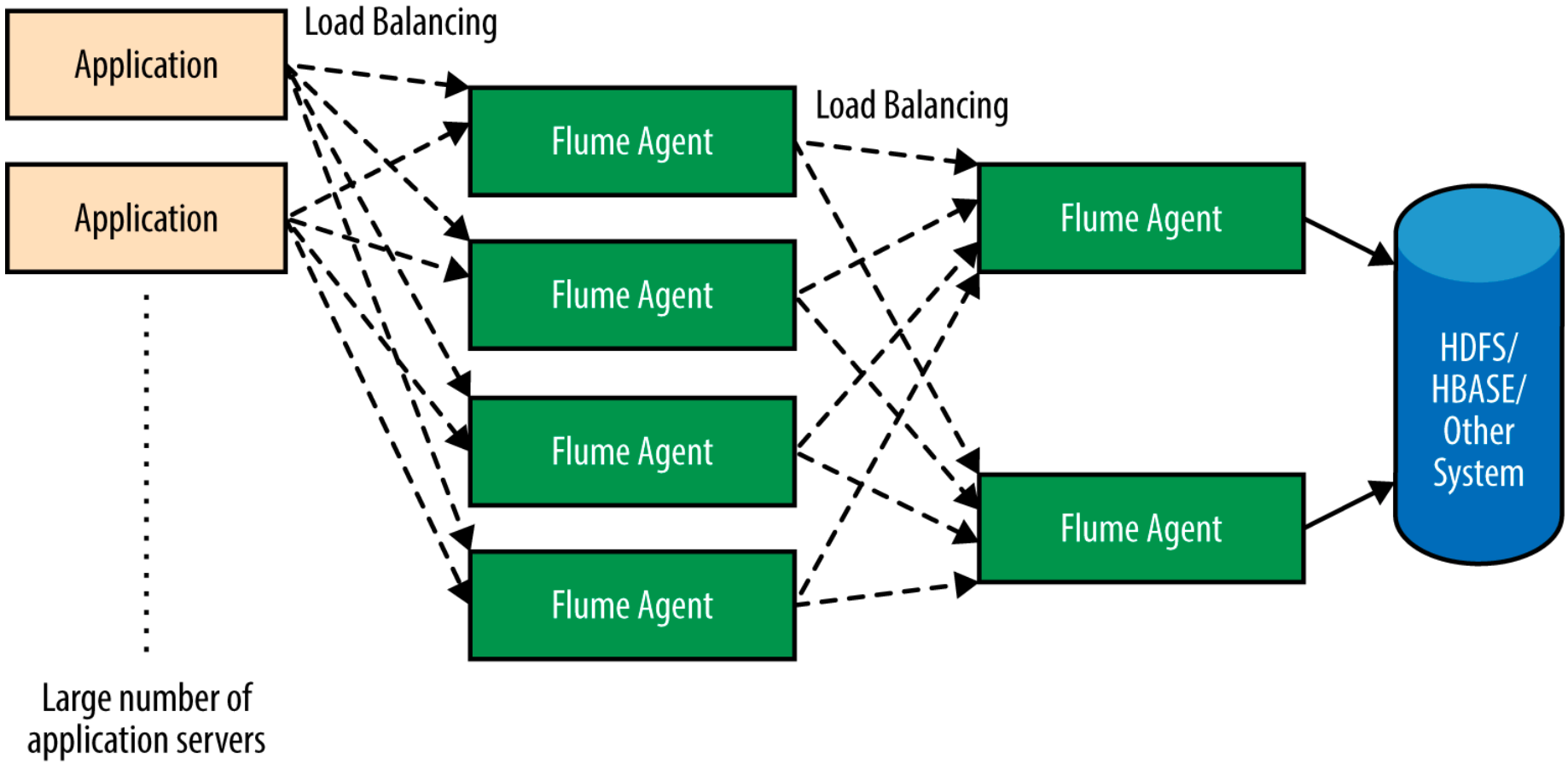


A Simple Flow

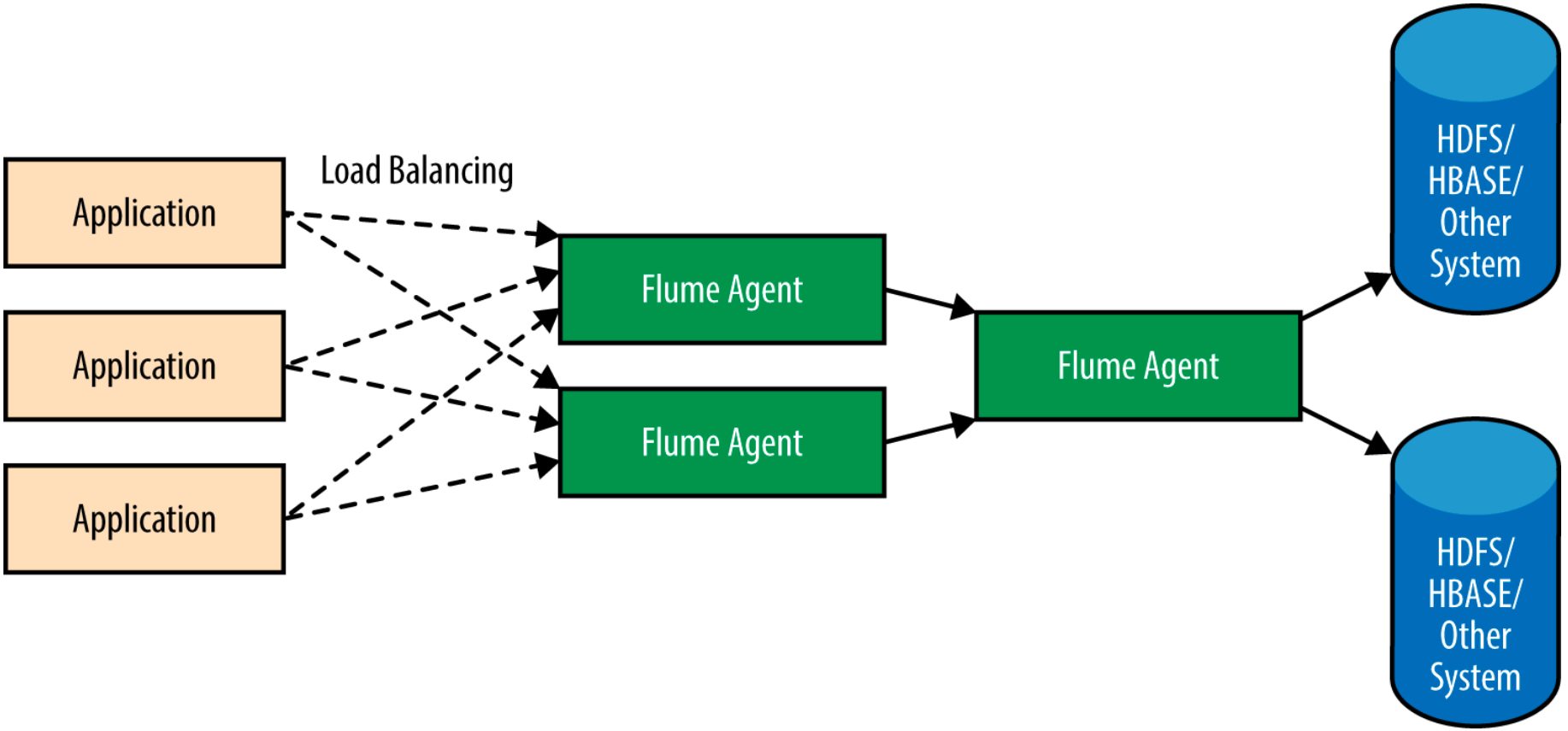
FLUJOS DE DATOS DE FLUME



FLUJOS DE DATOS DE FLUME



FLUJOS DE DATOS DE FLUME



DIFERENTES SOURCES

- Flume viene con varias sources:
  - HTTP, spooling, Avro RPC, syslog…

Table 3-1. Mandatory configuration parameters for all sources

Parameter	Description
`type`	The type of the source. This can be the FQCN or the alias of the source (only for sources that are part of Flume itself). The class must be installed using the *plugins.d* framework described in "Deploying Custom Code". The alias for each of Flume's built-in sources is mentioned in the relevant section.
`channels`	A space-separated list of channels the source should write events to. For all channels selected for writes by the channel selector based on the event and the routing parameters, the source will write events to the channels in the order specified in the configuration file. More details on this are in "Channel Selectors".

Table 3-2. Optional configuration parameters for all sources

Parameter	Description
`interceptors`	A list of names that represent a chain of interceptors
`interceptors.<interceptor_name>.*`	Parameters to pass to the interceptor with the specific name
`selector`	The alias or FQCN of the channel selector to use; if no selectors are specified, the *replicating channel selector* is used
`selector.*`	Configuration parameters to pass to the channel selector

DIFERENTES SOURCES

Table 3-3. Avro Source configuration

Config parameter	Default value	Description
type	-	The alias for an Avro Source is `avro`. The FQCN, which is `org.apache.flume.source.AvroSource` (case sensitive), can also be used.
bind	-	The IP address/hostname to bind to. To bind to all interfaces on the machine, use 0.0.0.0.
port	-	The port to bind to.
`threads`	`infinity`	The maxmium number of worker threads to accept incoming data from clients/Avro Sinks.
`ssl`	`false`	Should SSL be enabled? If this is set to `true`, all clients connecting to this source are required to use SSL. If SSL is enabled, the `keystore` and `keystore-password` parameters are required.
`keystore`	-	The path to the keystore to use for SSL. This is a required parameter if SSL is enabled.
`keystore-password`	-	The password to be used to open the keystore. This is a required parameter if SSL is enabled.
`keystore-type`	`JKS`	The type of keystore that is being used.
`compression-type`	-	The compression format used to decompress the incoming data. The only compression format supported is zlib [zlib_ch3]. To accept zlib-compressed data, set this parameter to `deflate`.

DIFERENTES SOURCES

Table 3-5. HTTP Source configuration

Config parameter	Default value	Description
type	-	The alias for the HTTP Source is `http`. The FQCN, which is `org.apache.flume.source.http.HttpSource` (case sensitive), can also be used.
bind	-	The IP address/hostname to bind to. To bind to all interfaces on the machine, use 0.0.0.0.
port	-	The port to bind to.
`enableSSL`	`false`	To enable SSL, this parameter should be set to `true`.
`keystore`	-	The path to the keystore file to be used.
`keystorePassword`	-	The password to be used for accessing the keystore.
`handler`	`JSONHandler`	The FQCN of the handler class that should be used by the HTTP Source to convert the HTTP request into Flume events. See "Writing Handlers for the HTTP Source*" to learn how to write handlers for the HTTP Source.
`handler.*`	-	Any parameters that have to be passed to the handler can be passed in through the configuration by using the `handler.` prefix.

DIFERENTES SOURCES

Table 3-6. Spooling Directory Source configuration

Parameter	Default	Description
type	-	The alias for the Spooling Directory Source is `spooldir`. The FQCN is `org.apache.flume.source.SpoolDirectorySource`.
spoolDir	-	The directory to "watch" and read files from. Subdirectories of this directory are not scanned.
batchSize	100	The maximum number of events to write to the channel per transaction.
ignorePattern	^$	Files with names matching this regex are ignored and data is not read from them.
deletePolicy	never	When to delete ingested files—must be `never` or `immediate`.
fileSuffix	.COMPLETED	The suffix to use for files that have been completely ingested. The "." is *required* if this is to be an extension.
fileHeader	false	If set to `true`, the filename is added to the event headers.
fileHeaderKey	file	The key to use in the headers, if the filename is added to headers.
trackerDir	.flumespool	The directory where the Spooling Directory Source stores the metadata that is used to restart the source from where it left off.
deserializer	line	The alias or FQCN of the `Builder` class that can be used to build the deserializer that should be used to read custom-formatted data. "Reading Custom Formats Using Deserializers*" explains how to write deserializers.
deserializer.*	-	Any parameters to be passed to the deserializer.
inputCharset	UTF-8	The character set to use when the deserializer calls `readChar`.

DIFERENTES SOURCES

Table 3-9. Syslog Source configuration

Parameter	Default	Description
type	-	The Syslog UDP Source type is `syslogudp` and the Multiport Syslog Source type is `multiport_syslogtcp`.
host	-	The IP address/hostname to bind to. To bind to all interfaces on the machine, use 0.0.0.0.
keepFields	false	If set to `true`, all fields from the syslog message are left in the event body in addition to having them in the event headers.

Table 3-10. Syslog UDP Source configuration

Parameter	Default	Description
port	-	The port to bind to.

Table 3-11. Multiport Syslog Source configuration

Parameter	Default	Description
ports	-	A space-separated list of ports to bind to.
portHeader	-	The port on which the event was received is included in the headers as the value of the key specified by this header. If this parameter is not configured, the port information is not included.
charset.default	UTF-8	The default character set to use.
charset.port.<port>	-	The character set to use for a specific port.
eventSize	2500	The maximum size of a single event, in bytes.
batchSize	100	The number of events to buffer in memory before writing to the channel.
readBufferSize	1024	The buffer size the underlying server should use.
numProcessors	-	The number of processors to use. This allows the source to increase the degree of parallelism.

DIFERENTES CHANNELS

- Flume viene con dos channels:
  - Memory Channel.
  - File Channel.

Table 4-1. Memory Channel configuration

Config parameter	Default value	Description
type	-	The alias for the Memory Channel is `memory`. The FQCN is `org.apache.flume.channel.MemoryChannel` (case sensitive).
`capacity`	`100`	The maximum number of committed events the channel can hold.
`transactionCapacity`	`100`	The maximum number of events that can be put or taken in a single transaction.
`byteCapacity`	80% of the total amount of heap space available to the process	The maximum amount of heap space (in bytes) this channel is allowed to use.
`byteCapacityBufferPercentage`	`20`	The percent of `byteCapacity` to consider keeping as a buffer between the byte capacity of the channel and the total size of the bodies of all events currently in the channel.
`keep-alive`	`3`	The maximum period of time (in seconds) each put or take should wait for completion.

DIFERENTES CHANNELS

Table 4-2. File Channel configuration

Config parameter	Default value	Description
type	-	The alias for the File Channel is `file`. The FQCN is `org.apache.flume.channel.file.FileChannel` (case sensitive).
`capacity`	`1000000`	The maximum number of committed events the channel can hold.
`transactionCapacity`	`1000`	The maximum number of events that can be put or taken in a single transaction.
`checkpointDir`	*~/flume/filechannel/checkpoint*	The directory to which the channel should write out the checkpoint.
`dataDirs`	*~/flume/filechannel/data*	A comma-separated list of directories to use to write the events to. Configuring multiple directories, each mounting a different disk, can dramatically improve performance by writing to the disks in parallel.
`useDualCheckpoints`	`false`	Tells the channel whether to back up the checkpoint once it has completely been written out. This must be either `true` or `false`. If this is set to `true`, **`backupCheckpointDir`** must be set.
`backupCheckpointDir`	-	The directory to back up the checkpoint to. If the primary checkpoint is corrupt or incomplete, the channel can recover from the backup, thus avoiding a full replay of the data files. This parameter should point to a directory different to **`checkpointDir`**.
`checkpointInterval`	`30`	The time period (in seconds) between consecutive checkpoints.
`maxFileSize`	`1623195647`	The maximum size (in bytes) of each data file. Once the file reaches this size (or will reach it once the next event is written to it), the file is rolled and a new data file is created in that directory. If this value is set to higher than the default value, the channel still uses the default as the maximum value.
`minimumRequiredSpace`	`524288000`	The minimum amount of space (in bytes) required on each volume for the channel to continue operation. If any one of the volumes on which the data directories is mounted has only this much space remaining, the channel will stop operation to prevent corruption and avoid incomplete data being written out. The minimum possible value for this parameter is `1048576` (1 MB).
`keep-alive`	`3`	The maximum period of time (in seconds) each put or take should wait for completion.

DIFERENTES SINKS

- Flume viene con varios sinks:
  - HDFS, HBase, Solr, Elastic Search, etc.: Llamados *terminal sinks*, ya que aparecen al final del flujo de flume.
  - RPC sinks: Utilizados para definir un flujo de datos donde un agente escribe y otro agente lee.
  - Sinks para testear: logger, rolling file…

Table 5-1. Mandatory configuration parameters for all sinks

Parameter	Description
type	The type of the sink. This can be the fully qualified class name or the alias of the sink (only for sinks that are part of Flume itself). The class must be in Flume's classpath. The alias for each of Flume's built-in sinks is mentioned in the relevant section.
channel	The channel to read events from.

Table 5-4. HDFS Sink configuration

Parameter	Default value	Description
type	-	The alias for the HDFS Sink is `hdfs`. The FQCN, which is `org.apache.flume.sink.hdfs.HDFSEventSink`, can also be used.
hdfs.path		The directory path this sink should write to. Escape sequences are allowed.
hdfs.filePrefix	FlumeData	The prefix of the filename. The filename will consist of this prefix followed by a number and then the name.
hdfs.fileSuffix	-	The suffix to use for the filenames. If extensions are required, the "." should be explicitly specified.
hdfs.inUsePrefix	-	The prefix to use for the filename while the HDFS Sink is writing to it.
hdfs.inUseSuffix	.tmp	The suffix to use for the filename while the HDFS Sink is writing to it.
hdfs.timeZone	-	The time zone to use for creating the bucket path.
hdfs.rollInterval	30	The time in seconds before a file is rolled. Set this to `0` to disable time-based file rolling.
hdfs.rollSize	1024	The number of events to be written into a file before it is rolled. Set this to `0` to disable event count–based file rolling.
hdfs.batchSize	100	The maximum number of events to write per batch.
hdfs.idleTimeout	0	The maximum time period in seconds to wait between consecutive events to a file before closing it. Setting this to `0` disables this.
hdfs.fileType	SequenceFile	The file format to use. This can be one of `SequenceFile`, `DataStream`, or `CompressedStream`.
hdfs.codeC	-	The compression codec to be used to compress the file. This can be one of `gzip`, `bzip2`, `lzo`, `lzop`, or `snappy`.
hdfs.maxOpenFiles	5000	The maximum number of files the HDFS Sink can keep open at a time.
hdfs.callTimeout	10000	The delay, in milliseconds, to wait before timing out each HDFS operation.
hdfs.threadsPoolSize	10	The number of threads in the pool that performs HDFS operations.
hdfs.rollTimerPoolSize	1	The number of threads in the pool that rolls HDFS files based on the `hdfs.rollInterval` and `hdfs.idleTimeout` parameters.
hdfs.kerberosPrincipal	-	The Kerberos principal to use to log in to the Kerberos key distribution center (KDC) (to be used with secure HDFS).
hdfs.kerberosKeytab	-	The path to the keytab file to use with `hdfs.kerberosPrincipal` to log in to the KDC (to be used with secure HDFS).
hdfs.proxyUser	-	The user Flume should impersonate, if required. If set to `none`, Flume writes data as the current user.
hdfs.useLocalTimeStamp	false	If set to `true`, the HDFS Sink will use the timestamp on the current agent to do time-based bucketing.
hdfs.round	false	Indicates whether the timestamp on the event should be rounded down.
hdfs.roundUnit	second	The unit of the `hdfs.roundValue` configuration parameter (can be `second`, `minute`, or `hour`).
hdfs.roundValue	1	The timestamp will be rounded to the largest multiple of this parameter in the units specified by the `hdfs.roundUnit` parameter.
serializer	TEXT	The serializer to use. Other built-in serializers can be used by setting this to `AVRO_EVENT` or `HEADER_AND_TEXT`. To use a custom class, this should be the FQCN of the `Builder` class of the serializer, which implements `EventSerializer$Builder`. The serializer itself must implement `EventSerializer`. See "Controlling the Data Format Using Serializers*".
serializer.*	-	Configuration parameters to be passed to the serializer. See "Controlling the Data Format Using Serializers*".

DIFERENTES SINKS

Table 5-12. Rolling File Sink configuration

Parameter	Default value	Description
type	-	The alias for the Rolling File Sink is `file_roll`. The FQCN, which is `org.apache.flume.sink.RollingFileSink`, can also be used.
`sink.batchSize`	`100`	The number of events the sink removes from the channel before the transaction is committed.
`sink.directory`	-	The directory that the sink should write events to.
`sink.rollInterval`	`30`	The time interval, in seconds, after which the file should be rolled.
`sink.serializer`	`TEXT`	The serializer to use to write events. This can be an alias for built-in serializers, or the FQCN for custom classes.
`sink.serializer.*`	-	The configuration parameters to be passed to the serializer.

Table 5-13. Logger Sink configuration

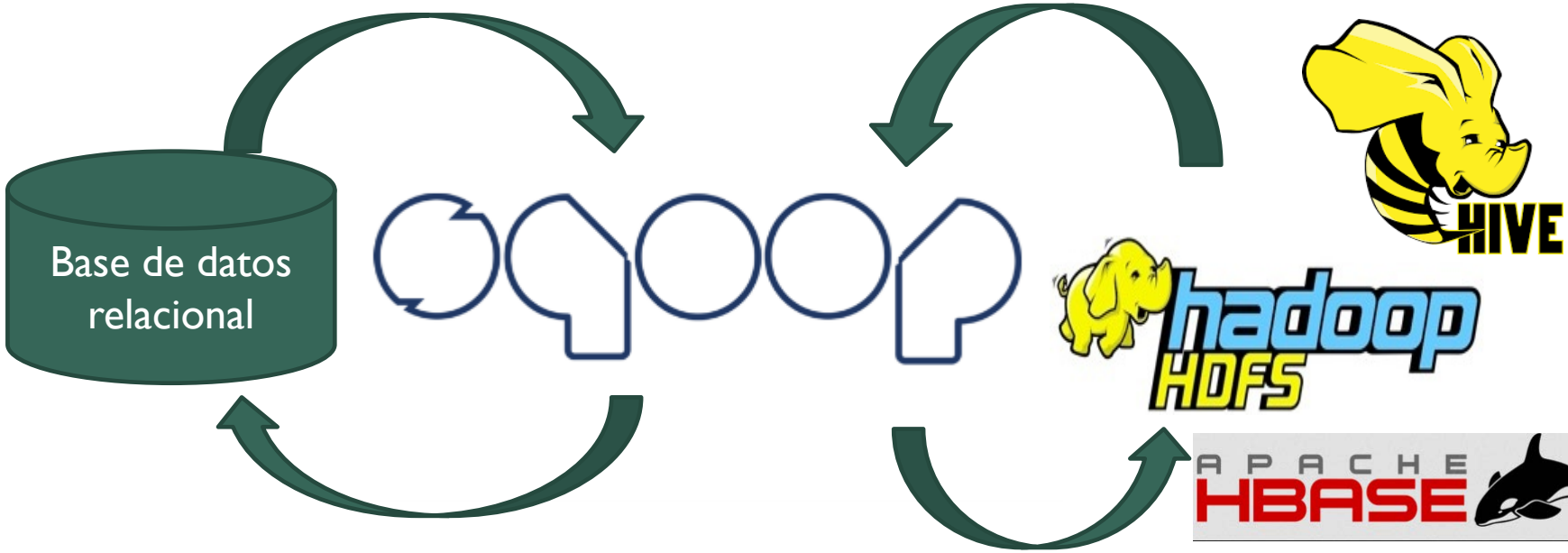
Parameter	Default value	Description
type	-	The alias for the Logger Sink is `logger`. The FQCN, which is `org.apache.flume.sink.LoggerSink`, can also be used.

TRABAJANDO CON FLUME

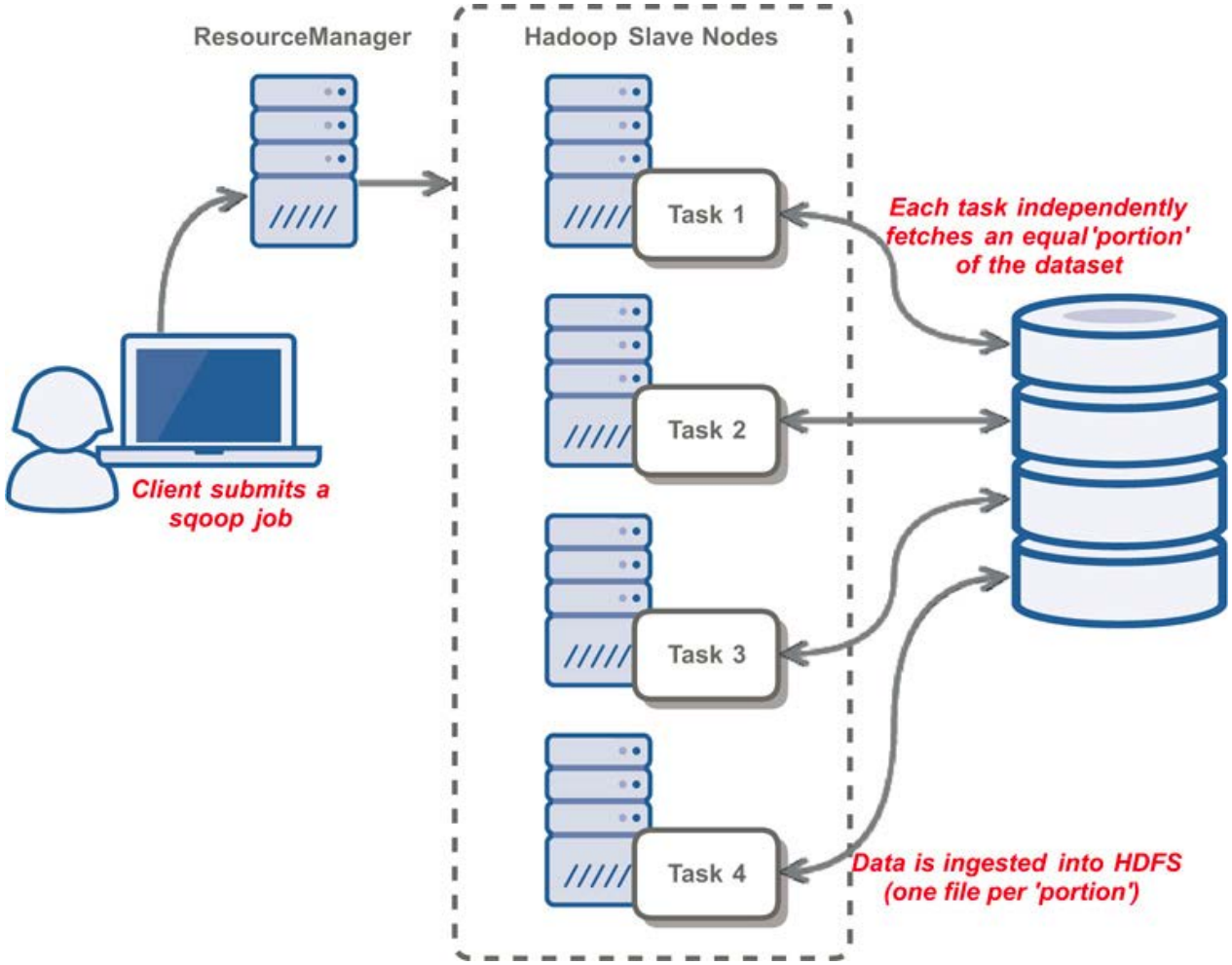
- Trabajo con Flume:
  - flume-estudiantes.ipynb

IMPORTANDO DE RDBMS: SQOOP

- Sqoop es una herramienta para realizar importaciones y exportaciones de datos entre los sistemas de almacenamiento de Hadoop y bases de datos relacionales.



SQOOP


ResourceManager
Hadoop Slave Nodes
Task 1
Task 2
Task 3
Task 4
Each task independently fetches an equal 'portion' of the dataset
Client submits a sqoop job
Data is ingested into HDFS (one file per 'portion')

SQOOP

- Comandos Sqoop:
  - sqoop help
    - codegen Generate code to interact with database records
    - create-hive-table Import a table definition into Hive
    - eval Evaluate a SQL statement and display the results
    - export Export an HDFS directory to a database table
    - help List available commands
    - import Import a table from a database to HDFS
    - import-all-tables Import tables from a database to HDFS
    - import-mainframe Import datasets from a mainframe server to HDFS
    - job Work with saved jobs
    - list-databases List available databases on a server
    - list-tables List available tables in a database
    - merge Merge results of incremental imports
    - metastore Run a standalone Sqoop metastore
    - version Display version information

SQOOP

- Importando:
  - sqoop help import
    - --connect <jdbc-uri> Specify JDBC connect
    - --username <username> Set authentication username
    - --password <password> Set authentication password
    - --table <table-name> Table to read for import
    - --as-parquetfile Imports data to Parquet files
    - --hive-import Import tables into Hive
    - --hive-overwrite Overwrite existing data in the Hive table
    - --hive-database <database-name> Sets the database name to use when importing to hive
    - --hive-table <table-name> Sets the table name to use when importing to hive
    - --hive-overwrite Overwrite existing data in the Hive table
    - …
  - Al menos, incluir --connect and --table

SQOOP

- Exportando:
  - sqoop help export
    - --connect <jdbc-uri> Specify JDBC connect string
    - --username <username> Set authentication username
    - --password <password> Set authentication password
    - --table <table-name> Table to populate → Debe existir
    - --export-dir <dir> HDFS source path for the export
    - …
  - Al menos hay que especificar --connect, --export-dir, and --table

TRABAJANDO CON SQOOP

- Trabajo con Sqoop:
  - sqoop-estudiantes.ipynb

Inyección de datos

Agustín C. Caminero Herráez
Depto. de Sistemas de Comunicación y Control.
UNED
accaminero@scc.uned.es / @accaminero